

Refactorización de `DiscretePotentialOperations`

Descomposición en clases cohesivas mediante el patrón Facade

`org.openmarkov.core` — paquete `potential.operation`

Manuel Arias Calleja

Marzo de 2026

Índice

1. Contexto y motivación	2
1.1. El papel de <code>DiscretePotentialOperations</code>	2
1.2. Problemas del diseño original	2
2. Patrón de diseño: Facade	3
2.1. Descripción del patrón	3
2.2. Por qué el patrón Facade encaja aquí	3
2.3. Principio complementario: Responsabilidad Única	4
3. Diagramas de clases	4
3.1. Antes de la refactorización	4
3.2. Después de la refactorización — estructura general	5
3.3. Después de la refactorización — detalle de métodos	6
4. Detalle de cada clase extraída	7
4.1. <code>TablePotentialArithmetic</code>	7
4.2. <code>TablePotentialElimination</code>	7
4.3. <code>TablePotentialMaximization</code>	7
4.4. <code>TablePotentialMerge</code>	8
4.5. <code>TablePotentialTransform</code>	8
4.6. <code>TablePotentialFactory</code>	8
5. Gestión del estado compartido y dependencias entre clases	8
5.1. La constante <code>maxRoundErrorAllowed</code>	8
5.2. Auxiliares compartidos en <code>AuxiliaryOperations</code>	9
6. Impacto en los llamadores existentes	9
7. Ventajas del nuevo diseño	9
7.1. Navegabilidad	9
7.2. Responsabilidad única y cohesión	9
7.3. Encapsulación forzada	9
7.4. Testabilidad	9
7.5. Reducción de la carga cognitiva	9
7.6. Evolución futura más sencilla	10
8. Resumen de los cambios	10

Resumen ejecutivo

La clase `DiscretePotentialOperations` había crecido con el tiempo hasta alcanzar aproximadamente 2 175 líneas de código, mezclando seis responsabilidades distintas en un único fichero monolítico: operaciones aritméticas, eliminación de variables, maximización, fusión de potenciales, transformaciones sobre un único potencial y métodos de fábrica. Esto dificultaba la navegación, el testing aislado y la evolución futura del código.

La refactorización descrita en este documento extrae esas seis responsabilidades en seis clases cohesivas de visibilidad de paquete, dejando `DiscretePotentialOperations` como una *facade* pura de unas 460 líneas. La API pública queda completamente inalterada: los 28 llamadores repartidos entre los módulos `core` e `inference` siguen compilando y pasando sus tests sin ninguna modificación.

1. Contexto y motivación

1.1. El papel de `DiscretePotentialOperations`

`DiscretePotentialOperations` es una clase de utilidades estáticas que proporciona todas las operaciones algebraicas y combinatorias primitivas sobre valores de tipo `TablePotential`. Se encuentra en el centro de cada algoritmo de inferencia de OpenMarkov: la eliminación de variables, la propagación de creencias, la resolución de diagramas de influencia y el análisis coste-efectividad invocan en última instancia uno o varios de sus métodos.

Por ser tan central, la clase había ido acumulando métodos a lo largo de los años. En el momento de esta refactorización contenía:

- 28 métodos públicos estáticos que abarcaban seis operaciones conceptualmente distintas.
- Aproximadamente 15 métodos privados auxiliares, muchos de los cuales eran invocados por métodos pertenecientes a grupos de responsabilidad diferentes.
- Una única constante pública (`maxRoundErrorAllowed`) referenciada no solo internamente sino también desde el módulo `inference`.
- Un total de aproximadamente 2 175 líneas de código.

1.2. Problemas del diseño original

Violación del Principio de Responsabilidad Única

Una clase debería tener un único motivo para cambiar. `DiscretePotentialOperations` tenía al menos seis: un cambio en el redondeo aritmético, un cambio en la política de desempate en la maximización, un cambio en la forma de fusionar potenciales... todos ellos requerirían editar el mismo fichero, aumentando el riesgo de regresiones accidentales y dificultando la revisión del código.

Navegabilidad deficiente

Con 2 175 líneas, localizar la implementación de una operación concreta exigía una búsqueda de texto completo o conocer de memoria el orden de las secciones dentro del fichero. La separación por comentarios ayudaba, pero no imponía ninguna frontera estructural real.

Mezcla de niveles de abstracción

La lógica de orquestación de alto nivel (por ejemplo, decidir qué variables conservar o eliminar) estaba entremezclada con la aritmética de índices de bajo nivel (por ejemplo, calcular los desplazamientos acumulados para recorrer tablas multidimensionales). El mismo método privado `findNextConfigurationAndIndexIncreasedVariable` era utilizado por las operaciones aritméticas, de eliminación, de maximización y de fusión, creando un acoplamiento implícito entre métodos que conceptualmente pertenecen a grupos distintos.

Dificultad para el testing incremental

Al residir toda la lógica en una sola clase, los tests unitarios no podían dirigirse con facilidad a un grupo de responsabilidad sin tocar potencialmente otro. Los fallos en los tests eran más difíciles de localizar.

2. Patrón de diseño: Facade

2.1. Descripción del patrón

El patrón **Facade** (Gamma et al., *Design Patterns*, 1994) proporciona una interfaz unificada y simplificada a un subsistema de clases. La facade no añade nueva funcionalidad; simplemente delega cada llamada a la clase de implementación apropiada, protegiendo a los clientes de la estructura interna del subsistema.

Listing 1: Delegación típica en la Facade

```
// Public facade -- API unchanged
public final class DiscretePotentialOperations {

    public static TablePotential multiply(
        List<TablePotential> potentials) {
        return TablePotentialArithmetic.multiply
            (potentials);
    }

    public static TablePotential
        multiplyAndMarginalize(
            Collection<TablePotential>
                potentials,
            List<Variable> variablesToKeep,
            List<Variable> variablesToEliminate)
        {
        return TablePotentialElimination
            .multiplyAndMarginalize(
                potentials,
                variablesToKeep,
                variablesToEliminate);
        }

    // ... 26 more delegating methods
}
```

2.2. Por qué el patrón Facade encaja aquí

El patrón Facade es especialmente adecuado en esta situación por tres razones:

1. **Existen 28 llamadores externos** repartidos entre dos módulos (**core** e **inference**) que ya usan `DiscretePotentialOperations` con su nombre actual. Renombrar o dividir la clase habría requerido actualizar todos esos puntos de llamada. La facade preserva la API existente exactamente.
2. **La implementación es compleja y debe ocultarse.** La aritmética de coordenadas multidimensionales, la gestión de desplazamientos y la lógica de desempate son detalles de implementación que los llamadores no necesitan conocer. La visibilidad de paquete de las clases de implementación impone esta encapsulación a nivel del compilador.
3. **El subsistema tiene una descomposición natural.** Los seis grupos de responsabilidad identificados en la clase original se corresponden directamente con seis clases de implementación cohesivas. Cada clase tiene un único motivo para cambiar.

2.3. Principio complementario: Responsabilidad Única

El patrón Facade gestiona la *interfaz externa*; el Principio de Responsabilidad Única (SRP) gobierna la *descomposición interna*. Cada una de las seis clases extraídas tiene exactamente una responsabilidad:

Clase	Responsabilidad
<code>TablePotentialArithmetic</code>	Multiplicar, sumar, dividir, evaluar funciones
<code>TablePotentialElimination</code>	Marginalizar, multiplicar y marginalizar
<code>TablePotentialMaximization</code>	Maximizar, multiplicar y maximizar
<code>TablePotentialMerge</code>	Fusionar potenciales para variables de decisión
<code>TablePotentialTransform</code>	Normalizar, proyectar, corregir distribuciones
<code>TablePotentialFactory</code>	Crear potenciales constantes, unidad o cero

3. Diagramas de clases

3.1. Antes de la refactorización

La figura 1 muestra el diseño original. Una única clase pública contiene todas las operaciones aritméticas, de eliminación, maximización, fusión, transformación y fábrica, junto con sus métodos privados auxiliares. La única otra clase del paquete relevante para este análisis es `AuxiliaryOperations`, que ya proporcionaba algunos auxiliares de bajo nivel.



Figura 1: Diagrama de clases antes de la refactorización. `DiscretePotentialOperations` contiene seis grupos de responsabilidad distintos y aproximadamente 15 métodos privados auxiliares en una única clase de 2 175 líneas.

La nota del diagrama destaca los cuatro problemas principales: el tamaño de la clase, el número de métodos públicos, los auxiliares privados que sirven a múltiples grupos no relacionados y la mezcla de seis responsabilidades conceptualmente distintas.

3.2. Después de la refactorización — estructura general

La figura 2 muestra la estructura de alto nivel tras la refactorización. `DiscretePotentialOperations` es ahora una *facade*: una clase de ≈ 460 líneas en la que cada método público es una llamada de delegación de una sola línea. Cada clase de implementación es de visibilidad de paquete, invisible para todos los llamadores externos.

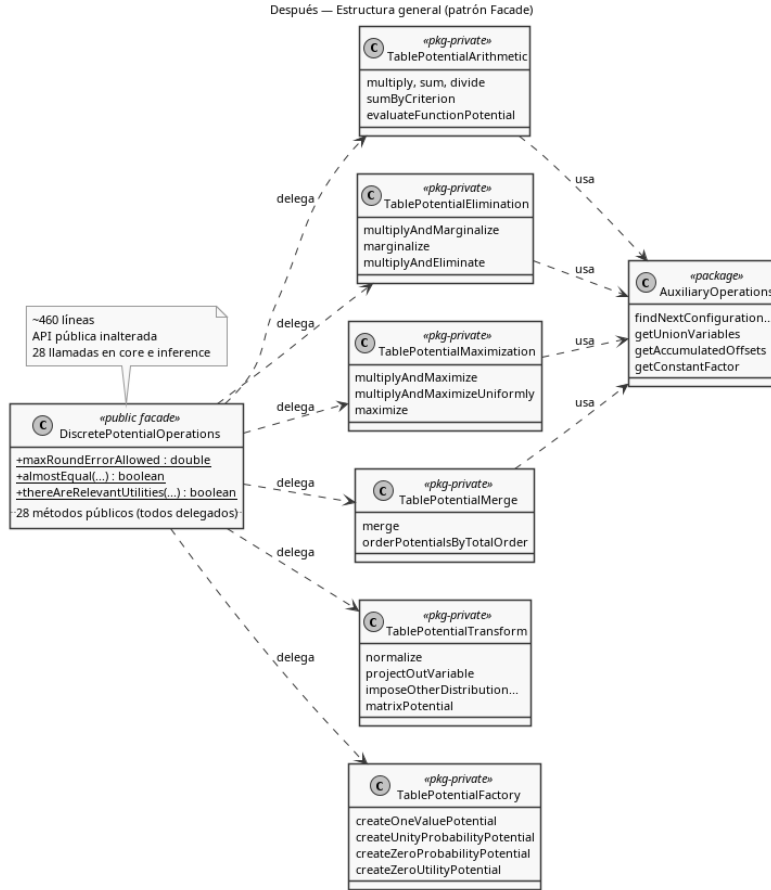


Figura 2: Estructura general tras la refactorización. `DiscretePotentialOperations` delega en seis clases de implementación cohesivas con visibilidad de paquete. `AuxiliaryOperations` proporciona la aritmética de índices compartida de bajo nivel.

Las flechas etiquetadas con *delega* representan relaciones de delegación: la facade invoca métodos de las clases de implementación, pero los llamadores externos al paquete solo ven la facade. Las flechas etiquetadas con *usa* representan usos ordinarios de los auxiliares compartidos en `AuxiliaryOperations`.

3.3. Después de la refactorización — detalle de métodos

La figura 3 muestra las firmas completas de los métodos de cada una de las seis nuevas clases con visibilidad de paquete. Todos los métodos tienen visibilidad de paquete (~), impuesta por la ausencia de modificador de acceso en el fuente Java, lo que significa que ninguna clase fuera del paquete `potential.operation` puede invocarlos directamente.

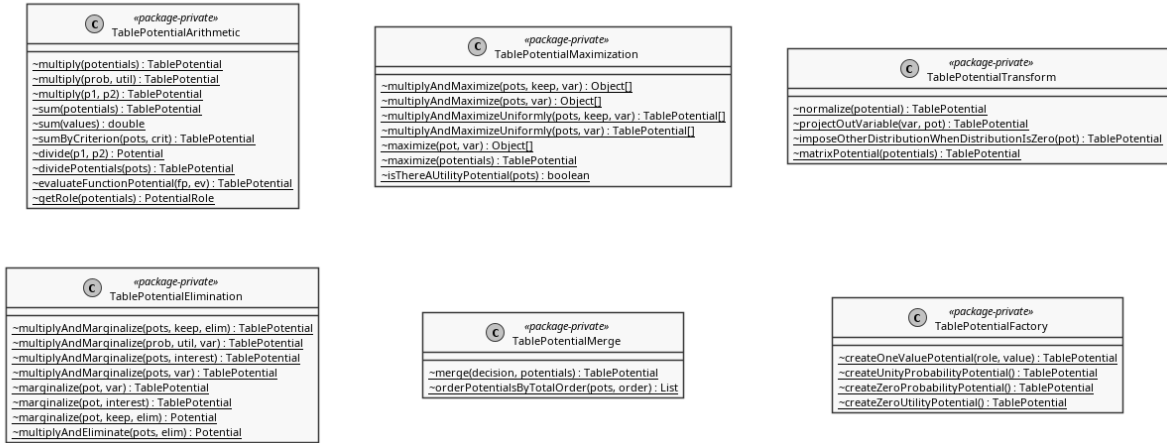


Figura 3: Listado detallado de métodos de las seis clases extraídas. El prefijo ~ en cada método denota visibilidad de paquete.

4. Detalle de cada clase extraída

4.1. TablePotentialArithmetic

Esta clase contiene la lógica más compleja: las operaciones `multiply` y `sum` sobre listas de `TablePotential` implican un bucle de incremento de coordenadas multidimensional, gestión de desplazamientos acumulados, separación del factor constante y tratamiento de `StrategyTree` para diagramas de influencia.

También contiene `evaluateFunctionPotential`, que proyecta un `FunctionPotential` (definido por una expresión simbólica) sobre una rejilla de valores `TablePotential`. Este método se trasladó aquí porque comparte los mismos auxiliares privados (`initializeCoordinates`, `buildResultPotential`, `findFirstNonNullCriterion`) que `multiply` y `sum`.

El auxiliar `getRole` determina el `PotentialRole` del resultado de una operación a partir de los roles de los potenciales de entrada, y es utilizado también por otras clases de implementación.

4.2. TablePotentialElimination

Contiene cuatro sobrecargas de `multiplyAndMarginalize` y tres de `marginalize`, cubriendo los casos habituales: eliminar una variable de un potencial, eliminar varias variables de una lista de potenciales y la sobrecarga especializada que multiplica un potencial de probabilidad por uno de utilidad mientras registra las intervenciones de `StrategyTree`.

`multiplyAndEliminate` es un envoltorio de conveniencia que calcula las variables a conservar como complemento de las variables a eliminar.

4.3. TablePotentialMaximization

Contiene la familia de métodos `multiplyAndMaximize`, utilizados durante la resolución de diagramas de influencia para calcular decisiones óptimas. La sobrecarga principal construye un `GTablePotential<Choice>` que registra, para cada configuración de las variables de condicionamiento, qué estado de la variable de decisión alcanza la máxima utilidad esperada (o todos los estados empatados, dentro de la tolerancia `maxRoundErrorAllowed`).

La variante `multiplyAndMaximizeUniformly` distribuye probabilidad uniformemente entre los

estados empatados en lugar de registrar un conjunto de elecciones, y se utiliza al calcular políticas óptimas aleatorizadas.

4.4. TablePotentialMerge

La operación `merge` combina una lista de potenciales (uno por estado de una variable de decisión) en un único potencial cuya primera variable es la variable de decisión. Gestiona tres flujos de datos en paralelo: los valores numéricos, las intervenciones de `StrategyTree` y las distribuciones `UncertainValue` para el análisis de incertidumbre del modelo.

`orderPotentialsByTotalOrder` es un auxiliar utilizado por el motor de inferencia para ordenar una lista de potenciales según un orden total dado de variables de decisión.

4.5. TablePotentialTransform

Reúne las operaciones que transforman un único potencial o producen uno estrechamente relacionado:

- `normalize`: normaliza una distribución de probabilidad condicional o conjunta.
- `projectOutVariable`: elimina una variable de decisión de un potencial proyectándola sobre su primer estado.
- `imposeOtherDistributionWhenDistributionIsZero`: reemplaza los cortes de todo ceros por una masa puntual degenerada en el primer estado, garantizando que la normalización posterior no divida por cero.
- `matrixPotential`: combina una lista mixta de potenciales de probabilidad y utilidad en un único potencial (producto de probabilidades por suma de utilidades), usado al construir representaciones matriciales de decisiones.

4.6. TablePotentialFactory

Una clase de fábrica simple con cuatro métodos estáticos para construir instancias de `TablePotential` que representan valores constantes: un potencial genérico de un valor, un potencial de probabilidad unidad (valor 1 con rol `CONDITIONAL_PROBABILITY`), un potencial de probabilidad cero y un potencial de utilidad cero. Se usan como elementos neutros en la multiplicación, como potenciales iniciales por defecto y en los fixtures de test.

`TablePotentialFactory` es la única clase nueva declarada `public` (en lugar de con visibilidad de paquete), ya que sus métodos de fábrica son invocados directamente desde varios puntos del código fuera del paquete `potential.operation`.

5. Gestión del estado compartido y dependencias entre clases

5.1. La constante `maxRoundErrorAllowed`

La constante de tolerancia `maxRoundErrorAllowed` no podía trasladarse a `AuxiliaryOperations` ni a `TablePotentialMaximization` porque es referenciada externamente por `CEPotentialOperation` en el módulo `inference` como `DiscretePotentialOperations.maxRoundErrorAllowed`. Por tanto permanece como campo `public static` en la facade.

`TablePotentialMaximization` accede a ella mediante el nombre completamente cualificado `DiscretePotentialOperations.maxRoundErrorAllowed`, lo que es correcto y deja clara la propiedad del campo.

5.2. Auxiliares compartidos en `AuxiliaryOperations`

El método de recorrido de índices multidimensionales `findNextConfigurationAndIndexIncreasedVariable` era un método privado de `DiscretePotentialOperations` utilizado por cuatro grupos de operaciones diferentes. Se trasladó a `AuxiliaryOperations` (que ya existía y albergaba auxiliares relacionados), donde ahora es accesible para todas las clases de implementación. La facade conserva un envoltorio público delegante para garantizar la compatibilidad con cualquier llamador externo existente.

6. Impacto en los llamadores existentes

La refactorización es **transparente para todos los llamadores**. Los 28 puntos de llamada identificados antes de la refactorización —repartidos entre `org.openmarkov.core` y `org.openmarkov.inference`— no requirieron **ninguna modificación**. Esto se verificó ejecutando la suite de tests completa de ambos módulos tras cada paso de extracción:

Módulo	Tests	Fallos
<code>org.openmarkov.core</code>	337	0
<code>org.openmarkov.inference</code>	13	0 (3 omisiones preexistentes)

7. Ventajas del nuevo diseño

7.1. Navegabilidad

Cada clase tiene un nombre autodescriptivo. Un desarrollador que busca la lógica de marginalización abre `TablePotentialElimination.java`; uno que depura un problema de maximización abre `TablePotentialMaximization.java`. Ya no hace falta desplazarse por 2000 líneas para encontrar el código relevante.

7.2. Responsabilidad única y cohesión

Cada una de las seis clases nuevas tiene un único motivo para cambiar. Una modificación en la política de desempate de `multiplyAndMaximize` queda contenida íntegramente en `TablePotentialMaximization`, sin riesgo de afectar accidentalmente a la lógica aritmética o de eliminación.

7.3. Encapsulación forzada

La visibilidad de paquete de las clases de implementación implica que ningún código externo al paquete `potential.operation` puede depender directamente de ellas. Todo acceso externo debe pasar por la facade, que ofrece una interfaz estable y documentada.

7.4. Testabilidad

Al estar cada clase centrada en una única preocupación, es posible escribir tests unitarios que apunten exactamente a una clase de implementación. Un test para `TablePotentialElimination` no necesita tener en cuenta la lógica de maximización ni de fusión, lo que simplifica los tests y facilita el diagnóstico de fallos.

7.5. Reducción de la carga cognitiva

La mayor de las seis clases nuevas, `TablePotentialArithmetic`, tiene aproximadamente 450 líneas, comparable al tamaño de la *nueva* facade. Ningún fichero supera las 400–450 líneas, lo que

mantiene cada fichero dentro de un rango que un desarrollador puede leer cómodamente en una sola sesión.

7.6. Evolución futura más sencilla

Los cambios futuros —como añadir aritmética acelerada por GPU o un nuevo tipo de operación sobre potenciales— pueden realizarse modificando o extendiendo una única clase cohesiva, añadiendo un método delegante a la facade y escribiendo tests dirigidos. El impacto de ese cambio queda limitado a un máximo de dos ficheros.

8. Resumen de los cambios

Fichero	Líneas	Cambio
DiscretePotentialOperations.java	≈460	Reducida a facade; eliminadas 3 constantes muertas
AuxiliaryOperations.java	ampliada	Añadidos 4 métodos procedentes de DPO
TablePotentialArithmetic.java	≈450	Nueva clase (visibilidad de paquete)
TablePotentialElimination.java	≈325	Nueva clase (visibilidad de paquete)
TablePotentialMaximization.java	≈400	Nueva clase (visibilidad de paquete)
TablePotentialMerge.java	≈280	Nueva clase (visibilidad de paquete)
TablePotentialTransform.java	≈150	Nueva clase (visibilidad de paquete)
TablePotentialFactory.java	≈55	Nueva clase (pública)